

MTH:2310 DISCRETE MATH FINAL REVIEW

- **Important Note**

- This review is **not** intended to cover every section from the semester, only to give recommendations on all material covered since Exam 3. **To review old material, refer back to the previous study guides.**

- **Recommended Preparation**

- Re-work examples from class.
- Work/re-work suggested problems from the book.
- Re-work quiz problems. Understand and learn from mistakes.
- Go over this study guide and make sure you know and understand every point on this study guide.
- Practice showing all your steps. Make solutions organized and be explicit/intentional in your writing.
- Replicate the exam environment. Can you do relevant problems with little to no help?

6.1 The Basics of Counting

- Be able to use the Sum Rule, Product Rule, and Subtraction Rule for counting.
- Know when each counting rule applies.
- Be able to use two or more of these rules in a single counting problem.
- Be able to explain your answer using a combination of words and diagrams.

Ex. Section 6.1, Exercises 7 - 12, 30 - 33, 50, 51

6.3 Permutations and Combinations

- Be able to determine whether unordered or ordered arrangements (or both) are being counted/considered.
- Be able to find all k -permutations or all k -combinations for a given set.
- Be able to count the total number of k -permutations of N objects.
- Be able to count the total number of k -combinations of N objects.
- Be able to express the total number of objects using the notation $P(n, k)$, $C(n, k)$, or $\binom{n}{k}$ when appropriate.

- Be able to explain your answer using a combination of words and diagrams.

Ex. Section 6.3, Exercises 11, 12, 14, 15, 21, 22, 33, 34, 37 - 39

Bonus. How many “trit” strings are there of length 8 and exactly 3 zeros?

Bonus. How many 10 digit positive integers with exactly 4 zeros are palindromes if no digit 1 - 9 may appear more than twice? *So then 1, 050, 220, 501 is included but 1, 010, 110, 101 is not included since there are 6 ones.*

6.2 Pigeonhole Principle

- Be able to use the Standard and Generalized Pigeonhole Principle.
- Be able to identify the “boxes” and “objects” given in a problem.

Ex. Section 6.2, Exercises 1, 2, 7, 17, 18

9.1 Relations and Their Properties

- Be able to show that a given relation on a set A is reflexive, symmetric, antisymmetric and/or transitive exhaustively or by using arbitrary elements of A .
- Be able to show that a given relation on a set A **fails to be** reflexive, symmetric, antisymmetric, and/or transitive by finding a counter example.
- Be able to represent any relation using a “bubble-arrow” diagram or a table.

Ex. Section 9.1, Exercises 1*, 2*, 3*, 6, 7

For problems marked with a (), additionally draw a “bubble-arrow” diagram and/or table corresponding to the relation.*

9.5 Equivalence Relations

- Know when a relation on a set A is an equivalence relation on A .
- Be able to show that a given relation is (or is not) an equivalence relation.
- Be able to use an equivalence relation R on a set A to identify distinct elements belonging to an equivalence class $[x]_R$ for a particular element $x \in A$.

Ex. Section 9.5, Exercises 1, 2, 11, 12, 25 - 29, 35 - 37

10.1 - 10.2 Graphs and Graph Terminology

- Be able to identify whether a given graph is a directed, mixed, undirected, simple, multi-, and/or pseudo- graph.
- Be able to identify all nodes belonging to the neighborhood of a single node, or subset of nodes within a graph.
- Be able to calculate the degree of a node in an undirected graph.
- Be able to calculate the in-degree and out-degree of a node in a directed graph.

Ex. Section 10.1, Exercises 3 - 9

Ex. Section 10.2, Exercises 1 - 4, 7 - 10

11.1 Introduction to Trees

- Be able to determine whether a given graph is a tree.
- Be able to form a rooted tree from a tree graph using a specified node as the root.

Ex. Section 11.1, Exercises 1, 2, 3*, 4*

For problems marked with a (), additionally draw the same tree but with root “f” and root “s” instead.*

11.3 Tree Traversal

- Be able to use the Universal Address System to create an ordered rooted tree.
- Be able to list the order in which nodes are visited using a **preorder traversal**, **inorder traversal**, or **postorder traversal**.

Ex. Section 11.3, Exercises 1 - 3, 6 - 15